TP555 - AI/ML

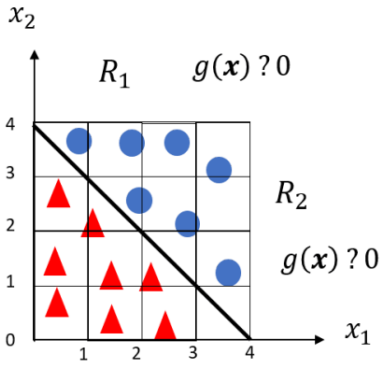
Lista de Exercícios #4

Classificação Linear: Parte 1

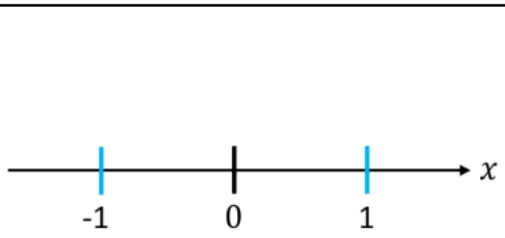
1. Dada a figura abaixo e a seguinte função discriminante:

$$g(x) = a_0 + a_1 x_1 + a_2 x_2,$$

encontre os pesos e as regiões de decisão.



2. Dada a seguinte figura, a qual representa os símbolos da modulação BPSK, encontre uma função discriminante linear, $g(\mathbf{x})$, que consiga classificar esses símbolos. Desenhe a função discriminante juntamente com os símbolos indicando as regiões de decisão.



bits	Símbolo
0	-1
1	$+1$

3. Dada a figura abaixo, a qual representa os símbolos da modulação QPSK, encontre as **funções discriminantes lineares**, que consigam classificar esses símbolos. Desenhe as funções discriminantes juntamente com os símbolos indicando as regiões de decisão. Em seguida, usando essas funções discriminantes faça o seguinte:

   a) Crie uma array com $N = 1000000$ valores aleatórios, variando entre 0 e 3, com a função `numpy.random.randint`, passe esse vetor para a função `modulator` e armazene a saída da função em um vetor `symbols`.

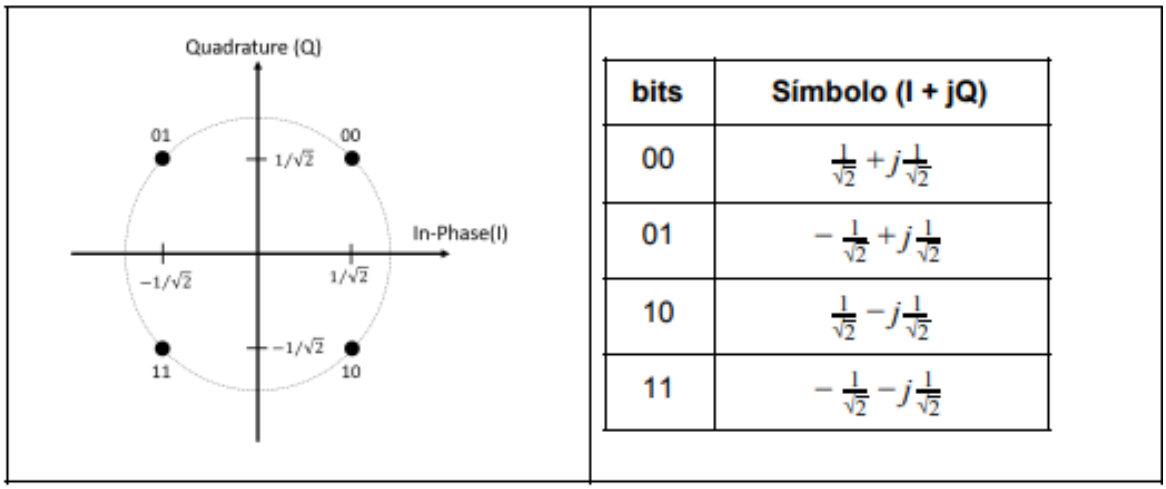
   b) Adicione ruído gaussiano branco ao vetor de saída da função `modulator`. Varie a relação energia de símbolo (E_s) por densidade espectral do ruído (N_0) de -2 a 20 dB em passos de 2 dB.

c) Crie uma função `demodulator` que utiliza as funções discriminantes e calcule o erro de símbolo simulado para cada valor de E_s/N_0.

d) Em seguida, plote um gráfico comparando a taxa de erro de símbolo (SER) simulada com a taxa de erro de símbolo teórica, a qual é dada por:

$$\text{SER} = \text{erfc}\left(\sqrt{\frac{E_s}{2N_0}}\right) - \frac{1}{4}\text{erfc}\left(\sqrt{\frac{E_s}{2N_0}}\right)^2.$$

(**Dica:** As duas curvas devem coincidir quase que perfeitamente.)



bits	Símbolo (I + jQ)
00	$\frac{1}{\sqrt{2}} + j\frac{1}{\sqrt{2}}$
01	$-\frac{1}{\sqrt{2}} + j\frac{1}{\sqrt{2}}$
10	$\frac{1}{\sqrt{2}} - j\frac{1}{\sqrt{2}}$
11	$-\frac{1}{\sqrt{2}} - j\frac{1}{\sqrt{2}}$

4. Neste exercício você utilizará o teorema de Bayes. Considere dois exames médicos, A e B, para um vírus. O teste A é 95% eficaz no reconhecimento do vírus quando ele está presente, mas tem uma taxa de falso positivo de 10% (indicando que o vírus está presente, quando ele não está). O teste B é 90% eficaz no reconhecimento do vírus, mas possui uma taxa de falso positivo de 5%. Os dois testes usam métodos independentes para identificar o vírus. 1% de todas as pessoas possuem o vírus. Digamos que uma pessoa é testada para o vírus usando apenas um dos testes e que o teste é positivo para o vírus. Qual teste, retornando positivo, é mais indicativo de alguém realmente estar com o vírus?

5. Neste exercício você vai prever se Jair pagará o empréstimo que ele está solicitando junto a um banco para montar uma indústria farmacêutica especializada na produção de hidroxicloroquina. Jair possui os seguintes atributos: **Possui casa própria? Não. Estado civil: Casado. Experiência de trabalho: 3**. Portanto, dado estes atributos sobre Jair, qual a probabilidade de que ele pague o empréstimo? Qual a probabilidade de que ele não pague o empréstimo? Baseado nas duas probabilidades, caso você trabalhasse no banco, você autorizaria o empréstimo? Para calcular as probabilidades, utilize os dados da tabela abaixo. (Dica: utilize a teoria do classificador naive Bayes).

   - OBS.: Todos os atributos são discretos, ou seja, assumem valores de um conjunto finito de valores. Por exemplo, o atributo experiência de trabalho assume apenas os seguintes valores: 0, 1, 2, 3, 4 e 5.

Possui Casa Própria?	Estado Civil	Experiência de Trabalho (0-5)	Pagou?
Sim	Solteiro	3	Sim
Não	Casado	4	Sim
Não	Solteiro	5	Sim
Sim	Casado	4	Sim
Não	Divorciado	2	Não
Não	Casado	4	Sim
Sim	Divorciado	2	Sim
Não	Casado	3	Não
Não	Casado	3	Sim
Sim	Solteiro	2	Não

6. Neste exercício você vai prever, baseado em alguns atributos físicos de uma pessoa, se ela é do sexo masculino ou feminino. Dado os seguintes atributos físicos de uma pessoa: altura = 1.83 metros, peso = 58.97 kg e tamanho do calçado = 20.32 cm. Baseado nas informações anteriores, qual classe tem maior probabilidade, ou seja, qual dos dois sexos teria a maior probabilidade? Para calcular as probabilidades, utilize os dados da tabela abaixo. **OBS.:** Apresente todos os cálculos feitos para se encontrar as probabilidades de cada classe, ou seja, neste exercício você não deve utilizar a biblioteca SciKit-learn.

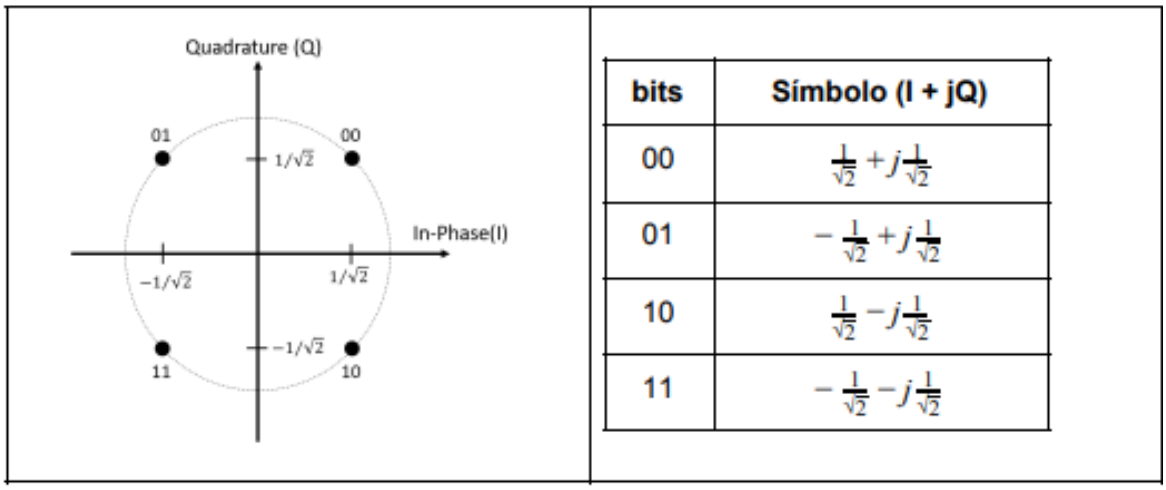
   (**Dica:** Assuma que os as probabilidades condicionais dos atributos seguem uma distribuição Gaussiana).

   (**Dica:** Assuma que a probabilidade da pessoa ser do sexo masculino ou do feminino é de 0.5, respectivamente).

   (**Dica:** utilize a teoria do classificador naive Bayes e lembre-se que o numerador da equação do classificador não influencia na maximização das probabilidades).

Altura [m]	Peso [kg]	Tamanho Calçado [cm]	Sexo
1,83	81,65	30,48	masculino
1,80	86,18	27,94	masculino
1,70	77,11	30,48	masculino
1,80	74,84	25,40	masculino
1,52	45,36	15,24	feminino
1,68	68,04	20,32	feminino
1,65	58,97	17,78	feminino
1,75	68,04	22,86	feminino

7. Neste exercício você irá implementar um classificador linear, utilizando o classificador naive Bayes, para realizar a detecção de símbolos QPSK. Os símbolos QPSK são dados pela figura e tabela abaixo.



bits	Símbolo (I + jQ)
00	$\frac{1}{\sqrt{2}} + j\frac{1}{\sqrt{2}}$
01	$-\frac{1}{\sqrt{2}} + j\frac{1}{\sqrt{2}}$
10	$\frac{1}{\sqrt{2}} - j\frac{1}{\sqrt{2}}$
11	$-\frac{1}{\sqrt{2}} - j\frac{1}{\sqrt{2}}$

O resultado do seu classificador (neste caso, um detector) pode ser comparado com a curva da taxa de erro de símbolo (SER) teórica, a qual é dada por

$$\text{SER} = \text{erfc}\left(\sqrt{\frac{E_s}{2N_0}}\right) - \frac{1}{4}\text{erfc}\left(\sqrt{\frac{E_s}{2N_0}}\right)^2.$$

Utilizando a classe GaussianNB do módulo `naive_bayes` da biblioteca sklearn, faça o seguinte:

a) Construa um classificador linear, utilizando o classificador naive Bayes, para realizar a detecção dos símbolos QPSK.
   i. Gere $N = 1000000$ símbolos QPSK aleatórios.
   ii. Passe os símbolos através de um canal AWGN.
   iii. Detecte a probabilidade de erro de símbolo para cada um dos valores do vetor $E_s/N_0 = [-2, 0, 2, 4, 6, 8, 10, 12, 14, 16, 18, 20]$.
   iv. Você pode utilizar o template abaixo para criar seu código

b) Apresente um gráfico comparando a SER simulada e a SER teórica versus os valores de E_s/N_0 definidos acima.

c) Podemos dizer que a curva simulada se aproxima da curva teórica da SER?

d) Se as classes, ou seja, os símbolos, tivessem probabilidades diferentes, nós poderíamos dizer que o classificador ML é equivalente ao MAP?

(**Dica:** Como os símbolos são representados por números complexos e a classe GaussianNB não suporta tal representação, você terá que instanciar 2 objetos da classe, um para cada componente do símbolo, ou seja, um classificador para a parte real (i.e., In-phase - I) e outro para a parte imaginária (Quadrature - Q).

(**Dica:** A função erfc pode ser importada da seguinte forma: from scipy.special import erfc).

(**Dica:** Uma rápida revisão sobre taxa de erro de símbolo pode ser encontrada no link:

http://www.dsplog.com/2007/11/06/symbol-error-rate-for-4-qam/

```
#Template of a QPSK detection loop
# Import all necessary libraries.
import numpy as np
from scipy.special import erfc
from sklearn.naive_bayes import GaussianNB
import matplotlib.pyplot as plt
# Number of QPSK symbols to be transmitted.
N = 1000000
# Instantiate a Gaussian naive Bayes classifier for each one of
   the parts
of a QPSK symbol.
gnb_re = ????
gnb_im = ????

# Create Es/N0 vector.
EsN0dB = np.arange(-2,22,2)
ser_simu = np.zeros(len(EsN0dB))
ser_theo = np.zeros(len(EsN0dB))
for idx in range(0,len(EsN0dB)):
    print('Es/N0 dB:', EsN0dB[idx])
    EsN0Lin = 10.0**(-(EsN0dB[idx]/10.0))

    # Generate N QPSK symbols.
    ip =(2.0 * (np.random.rand(N, 1) >= 0.5) - 1.0) +
    1j*(2.0 * (np.random.rand(N, 1) >= 0.5) - 1.0)
    # Normalization of energy to 1.
    s = (1/np.sqrt(2))*ip;

    # Generate noise vector with unitary variance.
    noise = np.sqrt(1.0/2.0)*(np.random.randn(N, 1) +
    1j*np.random.randn(N, 1))

    # Pass symbols through AWGN channel.
    y = s + np.sqrt(EsN0Lin)*noise

    # Fit model for real part.
    ????
    # Fit model for imaginary parts.
    ????
    # Prediction for real part.
    detected_ip_re = ????
    # Prediction for imaginary part.
    detected_ip_im = ????
    # Simulated QPSK BER.
    error_re = (ip.real != detected_ip_re)
    error_im = (ip.imag != detected_ip_im)
    error = 0;
    for i in range(0, N):
        if(error_re[i]==True or error_im[i]==True):
            error = error + 1
    ser_simu[idx] = 1.0 * error / N
    # Theoretical BPSK BER.
    ser_theo[idx] = erfc( np.sqrt(
       0.5*(10.0**(EsN0dB[idx]/10.0)) ) ) -
    (1/4)*(erfc(np.sqrt(0.5*(10.0**(EsN0dB[idx]/10.0)))))**2.0
```

8. Neste exercício você fará a classificação de textos em uma das categorias que serão definidas. Utilize como base o exemplo: `ClassifyingTextMultinomialNB.ipynb`. As categorias que devem ser classificadas pelo classificador são: `comp.windows.x`, `comp.os.ms-windows.misc`, `misc.forsale` e `rec.autos`. Treine e valide o classificador com os dados da base `20 Newsgroups corpus` da biblioteca `scikit-learn`. Plote a matriz de confusão e analise a matriz de confusão para responder:

   a) O que você percebe em relação à classe `comp.os.ms-windows.misc`?

   b) Qual uma possível explicação para o que você percebeu no item anterior?

   (**Dica:** Informações sobre matriz de confusão:

   https://en.wikipedia.org/wiki/Confusion_matrix

   https://dev.to/overridedeveloper/understanding-the-confusion-matrix-264i

9. Neste exercício você fará a classificação de algumas mensagens em duas categorias 'spam' e 'ham'. Utilize as 6 mensagens abaixo e seus respectivos rótulos para treinar um classificador naive Bayes Bernoulli.

```
# Features.

x_train = np.array(['free great offer if you join, a great offer for free!',
  'great offer for free delivery',
  'uber is promoting a great offer for free',
  'try uber for free for your 1st ride',
  'earn your uber 10 credit for free by applying for the uber visa credit card',
  'uber receipt'])


# Labels.

y_train = np.array(['spam','spam','spam','ham','ham','ham'])
```

   Use a classe CountVectorizer com o parâmetro binary=True para criar a matriz indicando a presença ou não de uma palavra, ou seja, uma matriz com valores booleanos. Em seguida, treine o classificador. De posse do modelo treinado, preveja a qual classe as 2 mensagens abaixo pertencem.

```
x_test = np.array(['Moonnight Trial', 'Limited offer: Free & Great
   Deal'])
```

   (**Dica:** use como base o exemplo: SPAMClassificationBernoulliNB.ipynb)

10. Neste exercício você irá comparar as classificações Multinomial e Bernoulli. Utilize as mensagens fornecidas e seus respectivos rótulos para treinar os classificadores naive Bayes com distribuições de Bernoulli e outro classificador naive Bayes com distribuição Multinomial.

```
x_train = np.array(['Chinese Beijing Chinese',
    'Chinese Chinese Shanghai',
    'Chinese Macao',
    'Tokyo Japan Chinese'])
y_train = np.array(['china','china','china','not china'])
```

Instancie um objeto da classe CountVectorizer com o parâmetro binary=True para o classificador naive Bayes com distribuição de Bernoulli. Para o classificador naive Bayes com distribuição Multinomial, instancie um objeto da classe CountVectorizer com o parâmetro binary=False. Em seguida, treine os classificadores. Utilize os seguintes comandos para verificar os nomes dos atributos e a matriz com a contagem dos atributos para cada instância da classe CountVectorizer, onde vect é o objeto da classe CountVectorizer e `x_train_dtm` é o matriz de contagem gerada pela execução do método `fit_transform` da classe CountVectorizer. Não se esqueça de transformar a mensagem de validação, `x_test`, com o método transform, antes de predizer sua classe para cada classificador.

```
print(vect.get_feature_names())
print(x_train_dtm.toarray())
```

De posse dos modelos treinados, pede-se

i. Imprima na tela de seu Notebook do Jupyter o nome dos atributos e a matriz de contagem dos atributos para cada uma das instâncias de **CountVectorizer**

ii. Utilize o método predict das classes BernoulliNB e MultinomialNB e preveja a qual classe a mensagem abaixo pertence para cada um dos classificadores.

```
x_test = np.array(['Chinese Chinese Chinese Tokyo Japan'])
```

iii. Calcule manualmente (ou seja, sem utilizar a biblioteca SciKit-learn) a probabilidade de cada classe, ou seja, 'china' e 'not china', dado a mensagem de teste para os 2 classificadores. Apresente os cálculos das probabilidades a priori e **a posteriori**.

iv. Utilize o método `predict_proba` das classes **BernoulliNB** e **MultinomialNB** para imprimir os resultados das probabilidades e confira se elas são iguais às que você encontrou manualmente no item (C). Utilize o comando abaixo para imprimir as probabilidades.

```
    print(model.predict_proba(x_test_dtm))
```

v. Como você deve ter percebido, existe diferença na classificação feita pelos 2 classificadore. Explique o motivo da classificação feita por cada classificador.

(**Dica:** Imprima o vetor de contagens de cada classificador com o comando,

```
     print(x_test_dtm.toarray())
```

compare as contagens de cada palavra no vetor, além disso, o item (C) acima vai te ajudar a entender e responder este item).

Observação: quando vocês forem calcular as probabilidades condicionais, vocês irão se deparar com probabilidades nulas, e.g., P('japan' | 'china') = 0, e isso faria com que as respostas finais fossem zeradas. Uma solução para esse problema é utilizar a **suavização de Laplace** também conhecida como **suavização adicione 1** [1,2]. A suavização é diferente para os 2 classificadores estudados, i.e., MultinomialNB e BernoulliNB

- **Suavização de Laplace para o caso do MultinomialNB:** Com a suavização as probabilidades condicionais se tornam

$$P(x_k|C_q) = \frac{contagem(x_k, C_q) + 1}{\sum_{l=1}^{K} contagem(x_l, C_q) + |V|}$$

onde $contagem(x_k, C_q)$ é número de vezes que a palavra x_k aparece entre todas as palavras que pertencem à classe C_q, $\sum_{l=1}^{K} contagem(x_l, C_q)$ é a soma total de palavras pertencentes à classe C_q e $|V|$ é o tamanho do vocabulário, ou seja, o número de atributos. Por exemplo, para o Classificador MultinomialNB $P(`japan'|`china') = (0+1)/(8+6)$, onde $contagem(x_k = `japan', C_q = `china') = 0$, $\sum_{l=1}^{K} contagem(x_l, C_q) = 8$ e $|V| = 6$, pois os atributos são 'beijing', 'chinese', 'japan', 'macao', 'shanghai' e 'tokyo', ou seja, 6 termos/palavras.

- **Suavização de Laplace para o caso do BernoulliNB:** Para o caso do BernoulliNB, utilizando-se a **suavização de Laplace**, $P(x_k|C_q)$ é calculada como

$$P(x_k|C_q) = \frac{M_{t,C_q} + 1}{M_c + 2}$$

onde M_{t,C_q} é o número de mensagens de treinamento da classe Cq que contém o termo/palavra x_k, enquanto M_C é o número total de mensagens de treinamento da categoria C_q e 2 pois existem dois casos a serem considerados para cada termo/palavra, ocorrência e não ocorrência.

Referências

[1] 'An Introduction to Naïve Bayes Classifier',
https://towardsdatascience.com/introduction-to-na%C3%AFve-bayes-classifier-fa59e3e24aaf

[2] 'Additive smoothing',
https://en.wikipedia.org/wiki/Additive_smoothing